

**Universidad Autónoma de Yucatán
Facultad de Matemáticas
Lic. en Ingeniería de Software**

TEORÍA DE LENGUAJES DE PROGRAMACIÓN

PROYECTO FINAL

MOTOR DE INFERENCIA LÓGICA COMO SERVICIO

Programación Lógica, Funcional y Asíncrona

Integrantes:

Juan Ángel Canche Góngora
Mauricio Antonio De Lázaro Lara
Christopher May Paat

Mayo 2026

1. Introducción

El presente proyecto implementa un Motor de Inferencia Lógica como Servicio (MLaaS): una API REST construida con Node.js, Express y Tau Prolog que permite ejecutar consultas lógicas contra una base de conocimientos escrita en lenguaje Prolog. Su desarrollo, en el marco de la materia Teoría de Lenguajes de Programación, tiene como propósito central demostrar la aplicación práctica de tres paradigmas fundamentales: la programación lógica, la programación funcional y la programación asíncrona.

Un motor de inferencia es un componente central en los sistemas de inteligencia artificial simbólica. Dado un conjunto de hechos y reglas que conforman la base de conocimientos, el motor es capaz de deducir nuevas conclusiones mediante razonamiento automático. A diferencia de los sistemas de aprendizaje automático, este enfoque simbólico garantiza que las inferencias sean correctas, trazables y explicables.

El proyecto expone esta capacidad de inferencia a través de un servicio HTTP, permitiendo que cualquier cliente —navegador, aplicación móvil u otro microservicio— realice consultas lógicas sin necesidad de integrar directamente un intérprete de Prolog en su entorno.

1.1 Tecnologías Utilizadas

- Node.js — Entorno de ejecución para JavaScript en el servidor, basado en el motor V8 de Chrome
- Express 5.2.1 — Framework web minimalista para construir la capa HTTP de la API REST
- Tau Prolog 0.3.4 — Intérprete de Prolog implementado íntegramente en JavaScript, sin dependencias nativas
- Nodemon 3.1.14 — Herramienta de desarrollo con reinicio automático del servidor ante cambios en el código

1.2 Problema que Resuelve

La integración de lógica declarativa en aplicaciones modernas suele requerir instalar entornos Prolog nativos (SWI-Prolog, GNU Prolog) y establecer comunicación entre procesos mediante sockets o llamadas al sistema. Este enfoque introduce dependencias nativas que complican el despliegue y la portabilidad.

Este proyecto elimina esa barrera al embeber el intérprete Tau Prolog directamente en el proceso de Node.js, exponiendo sus capacidades mediante una API REST estándar accesible desde cualquier plataforma.

1.3 Dominio de Aplicación

La base de conocimientos modela un sistema de gestión de contratos laborales con los siguientes elementos:

- Empleados: entidades que participan en contratos (juan, maria)
- Contratos: documentos formales con condiciones específicas (contrato1, contrato2)
- Condiciones de contrato: incumplimiento, demora, aprobación
- Reglas derivadas: penalidad aplicable, advertencia requerida, contrato válido

2. Paradigmas de Programación Utilizados

El proyecto integra deliberadamente tres paradigmas de programación complementarios. Desde la perspectiva de la teoría de lenguajes, cada paradigma corresponde a un modelo computacional distinto: el cálculo lambda para el funcional, la lógica de primer orden para el lógico, y el modelo de concurrencia por eventos para el asíncrono. La combinación de estos paradigmas produce una arquitectura más expresiva que cualquier enfoque aislado.

2.1 Programación Lógica

La base de conocimientos (knowledge_base.pl) está escrita en Prolog, el lenguaje más representativo de la programación lógica, cuyo fundamento teórico es la lógica de cláusulas definidas (Horn clauses). En este paradigma, los programas no especifican cómo calcular algo, sino qué es verdadero: la computación emerge del proceso de unificación y backtracking del motor de inferencia.

Hechos de la base de conocimientos:

```
% Empleados del sistema
empleado(juan).
empleado(maria).

% Contratos registrados
contrato(contrato1).
contrato(contrato2).

% Condiciones de los contratos
incumplimiento(contrato1).
demora(contrato2).
aprobado(contrato2).
```

Reglas de inferencia:

```
% Si un contrato tiene incumplimiento, aplica penalidad
penalidad_aplicable(X) :- incumplimiento(X).

% Si un contrato tiene demora, se requiere advertencia
advertencia_requerida(X) :- demora(X).

% Un contrato es valido si esta aprobado
contrato_valido(X) :- aprobado(X).
```

El motor Tau Prolog aplica resolución SLD (Selective Linear Definite clause resolution): busca unificaciones de manera sistemática en el orden de declaración de las cláusulas, explorando el árbol de búsqueda con backtracking automático para encontrar todas las soluciones posibles.

2.2 Programación Funcional

El servidor implementa un estilo funcional mediante funciones puras, composición de promesas y abstracción por funciones de orden superior. Estas técnicas tienen su

fundamento teórico en el cálculo lambda: las funciones son ciudadanos de primera clase que pueden pasarse como argumentos y componerse entre sí.

```
// Funcion pura: mismo input -> mismo output, sin efectos secundarios
function normalizarConsulta(query) {
  return query.trim().replace(/\s+/g, ' ');
}

// Promisificacion: abstraccion funcional sobre la API callback de Tau Prolog
function cargarBase(session, kb) {
  return new Promise((resolve, reject) => {
    session.consult(kb, { success: resolve, error: reject });
  });
}

function ejecutarQuery(session, query) {
  return new Promise((resolve, reject) => {
    session.query(query, { success: resolve, error: reject });
  });
}

function obtenerRespuesta(session) {
  return new Promise((resolve) => {
    session.answer({ success: resolve, fail: resolve, error: resolve });
  });
}
```

Las funciones `cargarBase`, `ejecutarQuery` y `obtenerRespuesta` son una cadena de transformaciones componibles: cada una recibe el resultado de la anterior, formando un pipeline funcional que convierte el API callback de Tau Prolog en un modelo de datos basado en Promises.

2.3 Programación Asíncrona

Node.js emplea un modelo de concurrencia basado en un event loop de un solo hilo. Para operaciones de I/O —como cargar la base de conocimientos y ejecutar consultas en Tau Prolog—, se utiliza el patrón `async/await`, que permite escribir código asíncrono con sintaxis síncrona sin bloquear el servidor.

```

app.post('/query', async (req, res) => {
  try {
    const { query } = req.body;
    if (!query) return res.status(400).json({ error: 'Consulta vacia' });

    const normalizedQuery = normalizarConsulta(query);
    const session = pl.create(1000);           // 1000 choice points
    await cargarBase(session, kb);             // Paso 1: cargar hechos
    await ejecutarQuery(session, normalizedQuery); // Paso 2: analizar
    consulta
    const answer = await obtenerRespuesta(session); // Paso 3: obtener
    respuesta

    res.json({ result: pl.format_answer(answer) });
  } catch (error) {
    res.status(500).json({ error: error.toString() });
  }
});

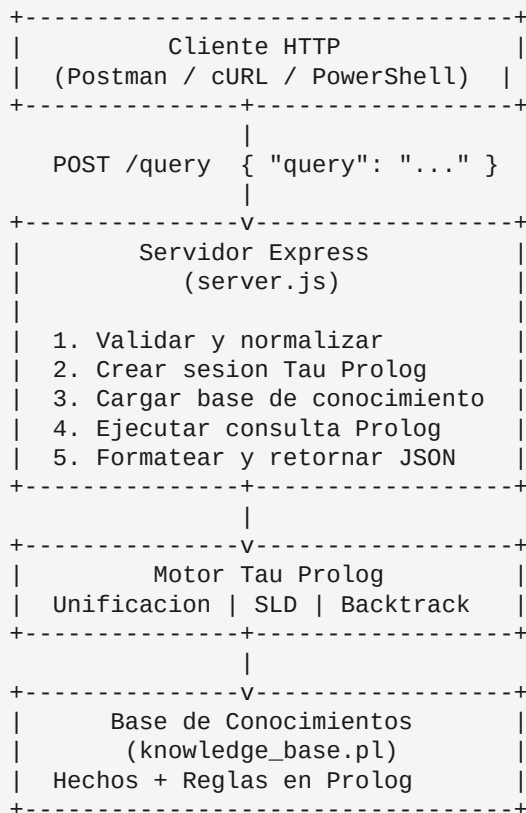
```

Este patrón permite que el servidor maneje múltiples solicitudes concurrentemente: mientras una solicitud aguarda la inferencia de Tau Prolog, el event loop puede procesar otras peticiones entrantes, sin necesidad de crear hilos adicionales del sistema operativo.

3. Arquitectura

3.1 Vista General del Sistema

El sistema se organiza en tres capas que siguen el principio de separación de responsabilidades: la capa de presentación (cliente HTTP), la capa de lógica de aplicación (Express + Tau Prolog) y la capa de datos (base de conocimientos Prolog).



3.2 Descripción de Componentes

server.js — Controlador Principal (74 líneas)

Punto de entrada de la aplicación. Inicializa Express con middleware JSON, lee la base de conocimientos del sistema de archivos al arrancar, define el endpoint POST / query con su manejador asíncrono, y provee las funciones auxiliares promisifyadas para interactuar con Tau Prolog. Incluye manejo de errores con códigos HTTP apropiados y logging de cada paso por consola.

knowledge_base.pl — Base de Conocimientos (27 líneas)

Archivo Prolog independiente que contiene los hechos y reglas del dominio. Su independencia de la capa de infraestructura es un diseño deliberado: puede modificarse y extenderse sin necesidad de tocar el código del servidor, respetando el principio de separación de responsabilidades.

package.json — Manifiesto del Proyecto

Define las dependencias de producción (express, tau-prolog), dependencias de desarrollo (nodemon) y los scripts de ejecución. El proyecto usa CommonJS (require/module.exports) por compatibilidad con Tau Prolog.

3.3 Flujo de Datos Completo

```
1. CLIENTE  -> POST http://localhost:3000/query
              Body: { "query": "penalidad_aplicable(contrato1)." }

2. EXPRESS  -> Parsea JSON, extrae campo "query"

3. SERVER   -> normalizarConsulta(): elimina espacios redundantes

4. SERVER   -> pl.create(1000): nueva sesion Prolog

5. SERVER   -> cargarBase(): session.consult(kb)
              Carga todos los hechos y reglas en la sesion

6. SERVER   -> ejecutarQuery(): session.query(query)
              Analiza sintaxis y prepara la resolucio

7. PROLOG    -> Resolucion SLD:
              ?- penalidad_aplicable(contrato1).
              -> Regla: penalidad_aplicable(X) :- incumplimiento(X).
              -> Unifica: X = contrato1
              -> Cuerpo: ?- incumplimiento(contrato1). -> HECHO: true

8. SERVER   -> obtenerRespuesta() -> pl.format_answer() -> "true ;"

9. CLIENTE  <- { "result": "true ;" }
```

3.4 Especificación de la API REST

Endpoint: POST /query
URL base: http://localhost:3000

Peticion:
Headers: Content-Type: application/json
Body: { "query": "<consulta Prolog>" }

Respuestas:
200 OK: { "result": "<respuesta formateada>" }
400 Bad Request: { "error": "La consulta no puede estar vacia" }
500 Server Err: { "error": "<mensaje de error Tau Prolog>" }

Formatos de resultado:
"true ;" Consulta satisfecha, mas soluciones disponibles
"false." Consulta insatisfecha
"X = val ;" Variable unificada con valor concreto

4. Ejemplo de Ejecución

4.1 Instalación y Arranque del Servidor

```
# Instalar dependencias
$ npm install

# Iniciar en modo produccion
$ npm start

> motor-inferencia-logica@1.0.0 start
> node server.js
Servidor escuchando en http://localhost:3000
```

4.2 Verificación de un Hecho Simple

La consulta más básica verifica si un hecho existe directamente en la base de conocimientos.

Solicitud:

```
POST http://localhost:3000/query
Content-Type: application/json

{ "query": "empleado(juan)." }
```

Proceso de inferencia:

```
?- empleado(juan).
   -> empleado(juan). [ENCONTRADO]
Resultado: true
```

Respuesta:

```
{ "result": "true ;" }
```

4.3 Consulta con Variable y Backtracking

Al utilizar una variable Prolog (identificada por iniciar con mayúscula), el motor busca todas las unificaciones posibles. El carácter ";" en la respuesta indica que existen más soluciones mediante backtracking.

Solicitud:

```
POST http://localhost:3000/query
Content-Type: application/json

{ "query": "empleado(X)." }
```

Proceso de inferencia:

```
?- empleado(X).  
  -> empleado(juan).  X = juan    [PRIMERA SOLUCION]  
  -> empleado(maria). X = maria   [DISPONIBLE VIA BACKTRACKING]  
Retorna primera solucion. ";" indica que hay mas.
```

Respuesta:

```
{ "result": "X = juan ;" }
```

4.4 Inferencia Mediante Reglas — Caso Central

Este caso ilustra el poder de la inferencia lógica: Tau Prolog deriva una conclusión que no está almacenada explícitamente, aplicando una regla sobre los hechos existentes.

Solicitud:

```
POST http://localhost:3000/query  
Content-Type: application/json  
  
{ "query": "penalidad_aplicable(contrato1)." }
```

Resolución SLD paso a paso:

```
?- penalidad_aplicable(contrato1).  
  
Paso 1: Busca penalidad_aplicable/1 -> es cabeza de regla:  
        penalidad_aplicable(X) :- incumplimiento(X).  
  
Paso 2: Unifica: X = contrato1  
  
Paso 3: Debe probar el cuerpo:  
        ?- incumplimiento(contrato1). -> HECHO ENCONTRADO  
  
Conclusion: penalidad_aplicable(contrato1) es VERDADERO
```

Respuesta:

```
{ "result": "true ;" }
```

4.5 Consulta Negativa — Closed World Assumption

Cuando la consulta no puede satisfacerse con ningún hecho ni regla, Tau Prolog aplica la Closed World Assumption: lo que no puede probarse se asume falso.

Solicitud:

```
POST http://localhost:3000/query  
Content-Type: application/json  
  
{ "query": "empleado(pedro)." }
```

Pedro no existe en ningún hecho. Tau Prolog agota todas las posibilidades y concluye:

```
{ "result": "false." }
```

4.6 Ejecución desde PowerShell (Windows)

```
# Hecho simple
$body = @{ query = 'empleado(juan).' } | ConvertTo-Json -Compress
Invoke-RestMethod -Method Post -Uri "http://localhost:3000/query" `
  -ContentType "application/json" -Body $body
# result: true ;

# Inferencia por regla
$body = @{ query = 'penalidad_aplicable(contrato1).' } | ConvertTo-Json -
Compress
Invoke-RestMethod -Method Post -Uri "http://localhost:3000/query" `
  -ContentType "application/json" -Body $body
# result: true ;

# Consulta negativa
$body = @{ query = 'empleado(pedro).' } | ConvertTo-Json -Compress
Invoke-RestMethod -Method Post -Uri "http://localhost:3000/query" `
  -ContentType "application/json" -Body $body
# result: false.
```

5. Conclusiones y Extensiones

5.1 Conclusiones

Este proyecto demuestra con éxito la integración de tres paradigmas de programación en un sistema cohesivo y funcional. El resultado es un microservicio de inferencia lógica completamente operativo que puede integrarse en cualquier arquitectura web moderna.

Logros Técnicos

- Integración exitosa de Prolog en Node.js mediante Tau Prolog sin dependencias nativas ni procesos externos
- API REST completamente funcional que expone capacidades de inferencia lógica a cualquier cliente HTTP
- Diseño asíncrono que maneja múltiples solicitudes concurrentemente sin bloquear el servidor
- Separación clara entre lógica de dominio (`knowledge_base.pl`) e infraestructura (`server.js`)
- Base de conocimientos extensible: nuevos hechos y reglas se integran sin modificar el servidor